# MCP TOOLS

# Printix MCP Server — Tool Reference

> **Version:** 6.8.10 **Transport:** Streamable HTTP ( `/mcp` for claude.ai) + SSE ( `/sse` for ChatGPT)
> **Auth:** OAuth 2.0 Bearer Token per tenant **Public Endpoint:** `https://mcp.printix.cloud`

This document lists every MCP tool exposed by the Printix MCP Server. Tools are grouped by feature area. All tools return a JSON-encoded string.

---

## Table of Contents


1. Status & Info
2. Printers & Queues
3. Print Jobs
4. Card Management (Printix Cloud)
5. Card Management (Local DB)
6. Users
7. Groups
8. Workstations
9. Sites & Networks
10. SNMP Configurations
11. Reporting — Queries
12. Reporting — Templates & Schedules
13. Reporting — Design & Preview
14. Demo Data
15. Audit Log & Feature Requests
16. Backup Management
17. Capture Profiles
18. Site & Network Aggregations
19. Cross-Source Insights *(v6.7.107)*
20. Card Management — Tenant-Wide *(v6.7.107)*
21. Print Jobs & Reporting — High-Level *(v6.7.107)*

22. Access & Governance *(v6.7.107)*
23. Agent Workflow Helpers *(v6.7.107)*
24. Quality of Life *(v6.7.107)*

---

## Status & Info

| Tool | Description |
|---|---|
| `printix_status()` | Returns server version, configured tenant, OAuth app health. Sanity-check tool. |

---

## Printers & Queues

| Tool | Description |
|---|---|
| `printix_list_printers(search, page, size)` | List printers in tenant. Supports free-text search. |
| `printix_get_printer(printer_id, queue_id)` | Full detail of a single printer/queue pair. |

---

## Print Jobs

| Tool | Description |
|---|---|
| `printix_list_jobs(queue_id, page, size)` | Current print jobs. Optionally filtered by queue. |
| `printix_get_job(job_id)` | Detail of one print job. |
| `printix_submit_job(...)` | Submit a new print job (3-step flow: submit → upload → complete). |
| `printix_complete_upload(job_id)` | Finalize a job after file upload. |
| `printix_delete_job(job_id)` | Cancel / delete a print job. |
| `printix_change_job_owner(job_id, new_owner_email)` | Transfer Secure-Print ownership to another user. |

---

## Card Management (Printix Cloud)

| Tool | Description |
|---|---|
| `printix_list_cards(user_id)` | All cards of one user, **enriched** with local DB mappings (raw UID, profile, notes, decoded hex/decimal). |
| `printix_search_card(card_id, card_number)` | Lookup a single card by Printix ID or physical number. |
| `printix_register_card(user_id, card_number)` | Register a card directly via Printix API (no local mapping). |
| `printix_delete_card(card_id)` | Remove a card from Printix. |

---

## Card Management (Local DB)

| Tool | Description |
|---|---|
| `printix_list_card_profiles()` | All transform profiles (built-in vendors + tenant-custom). |
| `printix_get_card_profile(profile_id)` | Single profile detail incl. rules JSON. |
| `printix_search_card_mappings(search, printix_user_id)` | Full-text search across local mappings. |
| `printix_get_card_details(card_id, card_number)` | Enriched single-card view combining Printix + local data. |
| `printix_decode_card_value(card_value)` | Decode a raw Printix base64 secret into hex / decimal / reversed variants. |
| `printix_transform_card_value(card_value, ...)` | Apply a transform profile to a raw UID — dry run. |
| `printix_get_user_card_context(user_id)` | One-shot: user + all cards (enriched) + all local mappings. |

---

## Users

| Tool | Description |
|---|---|
| `printix_list_users(role, query, page, page_size)` | List users. Default `role='USER'` — call again with `'GUEST_USER'` for guests. |
| `printix_get_user(user_id)` | Full user detail. |
| `printix_create_user(email, display_name, pin, password)` | Create a guest account. |
| `printix_delete_user(user_id)` | Delete a (guest) user. |
| `printix_generate_id_code(user_id)` | Issue a fresh 6-digit ID code for card-reader fallback. |

## Groups

| Tool | Description |
|---|---|
| `printix_list_groups(search, page, size)` | List groups. |
| `printix_get_group(group_id)` | Single group detail. |
| `printix_create_group(name, external_id)` | Create a group. |
| `printix_delete_group(group_id)` | Delete a group. |

## Workstations

| Tool | Description |
|---|---|
| `printix_list_workstations(search, page, page_size)` | List registered endpoints (Printix Client installations). |
| `printix_get_workstation(workstation_id)` | Full workstation detail. |

## Sites & Networks

| Tool | Description |
|---|---|
| `printix_list_sites(search, page, size)` | List sites. |
| `printix_get_site(site_id)` | Site detail. |
| `printix_create_site(name, path, ...)` | Create a site. |
| `printix_update_site(site_id, ...)` | Update a site. |
| `printix_delete_site(site_id)` | Delete a site. |
| `printix_list_networks(site_id, page, size)` | List networks (optionally scoped to a site). |
| `printix_get_network(network_id)` | Network detail. |
| `printix_create_network(name, ...)` | Create a network. |
| `printix_update_network(network_id, ...)` | Update network. |
| `printix_delete_network(network_id)` | Delete a network. |

## SNMP Configurations

| Tool | Description |
|---|---|
| `printix_list_snmp_configs(page, size)` | List SNMP configs. |
| `printix_get_snmp_config(config_id)` | Single SNMP config. |
| `printix_create_snmp_config(name, ...)` | Create SNMP config. |
| `printix_delete_snmp_config(config_id)` | Delete SNMP config. |

## Reporting — Queries

| Tool | Description |
|---|---|
| `printix_reporting_status()` | Health of the SQL Server data source + last ingest time. |
| `printix_query_print_stats(...)` | Raw print-volume aggregation (group by user/printer/site/etc.). |
| `printix_query_cost_report(...)` | Cost calculation per unit with configurable €/page. |
| `printix_query_top_users(start_date, end_date, top_n, metric, ...)` | Ranking of most active users. |
| `printix_query_top_printers(start_date, end_date, top_n, metric, ...)` | Ranking of most-used printers. |
| `printix_query_anomalies(...)` | Detect usage spikes / drops. |
| `printix_query_trend(...)` | Time series grouped by day/week/month. |
| `printix_query_any(sql_like_spec)` | Generic query escape hatch — execute a saved template spec. |

## Reporting — Templates & Schedules

| Tool | Description |
|---|---|
| `printix_save_report_template(...)` | Persist a configured report for re-use. |
| `printix_list_report_templates()` | All saved report templates. |
| `printix_get_report_template(report_id)` | Template detail. |
| `printix_delete_report_template(report_id)` | Delete template. |
| `printix_run_report_now(report_id, report_name)` | Execute a template ad hoc and return results. |
| `printix_send_test_email(recipient)` | Verify SMTP / email delivery. |
| `printix_schedule_report(...)` | Schedule a template (cron-like). |
| `printix_list_schedules()` | All active schedules. |
| `printix_delete_schedule(report_id)` | Remove schedule. |
| `printix_update_schedule(...)` | Modify schedule. |

## Reporting — Design & Preview

| Tool | Description |
|---|---|
| `printix_list_design_options()` | Available chart/layout presets for PDF generation. |
| `printix_preview_report(...)` | Render a report preview (HTML/PDF) without sending email. |

## Demo Data

| Tool | Description |
|---|---|
| `printix_demo_setup_schema()` | Create demo tables in SQLite if missing. |
| `printix_demo_generate(...)` | Generate synthetic print-job data (useful for dashboards without real data). |
| `printix_demo_rollback(demo_tag)` | Remove a demo batch by tag. |
| `printix_demo_status()` | Active demo datasets. |

## Audit Log & Feature Requests

| Tool | Description |
| --- | --- |
| `printix_query_audit_log(start_date, end_date, action_prefix, object_type, limit)` | Query the tenant audit log (user actions, credential changes, feature requests). |
| `printix_list_feature_requests(status, limit)` | List user-submitted feature requests / ideas. |
| `printix_get_feature_request(ticket_id)` | Detail of a specific request. |

## Backup Management

| Tool | Description |
| --- | --- |
| `printix_list_backups()` | Existing DB/secret backups under `/config/backups`. |
| `printix_create_backup()` | Create a fresh backup. |

## Capture Profiles

| Tool | Description |
| --- | --- |
| `printix_list_capture_profiles()` | Capture webhook profiles (OCR/forwarding pipelines). |
| `printix_capture_status()` | Capture subsystem health + recent webhook activity. |

## Site & Network Aggregations

| Tool | Description |
|---|---|
| `printix_site_summary(site_id)` | Site overview: networks + printers + SNMP. |
| `printix_network_printers(network_id, site_id)` | Printers on a specific network. |
| `printix_get_queue_context(queue_id, printer_id)` | Full context for a queue (printer, site, network, SNMP). |
| `printix_get_network_context(network_id)` | Network + its site + printers + SNMP in one call. |
| `printix_get_snmp_context(config_id)` | SNMP config + networks/sites/printers that use it. |

---

## Cross-Source Insights

*(new in v6.7.107)*

| Tool | Description |
|---|---|
| `printix_find_user(query)` | Fuzzy search across email/name/ID. Pre-step for anything needing a `user_id`. |
| `printix_user_360(query)` | Complete profile: user + cards (enriched) + workstations. |
| `printix_printer_health_report()` | Online/offline breakdown, grouped by site, with offline-list. |
| `printix_tenant_summary()` | Executive dashboard: counts for users, printers, workstations, groups, local cards. |
| `printix_diagnose_user(email)` | Heuristic troubleshooting — finds missing cards, role mismatches, orphaned accounts. |

---

## Card Management — Tenant-Wide

*(new in v6.7.107)*

| Tool | Description |
|---|---|
| `printix_list_cards_by_tenant(status)` | All cards across all users. Filter `mapped` / `unmapped` / `all`. |
| `printix_find_orphaned_mappings()` | Local mappings with no corresponding Printix card (leftovers from outside-app deletes). |
| `printix_bulk_import_cards(csv_data, profile_id, dry_run)` | CSV mass import ( `email,card_uid[,notes]` ) with transform profile. |
| `printix_suggest_profile(sample_uid)` | Tries all profiles against a sample UID and scores best match. |
| `printix_card_audit(user_email)` | Per-user card audit trail with source, notes, timestamps. |

---

## Print Jobs & Reporting — High-Level

*(new in v6.7.107)*

| Tool | Description |
|---|---|
| `printix_top_printers(days, limit, metric)` | Most-used printers in last N days (convenience wrapper). |
| `printix_top_users(days, limit, metric)` | Most-active users in last N days (convenience wrapper). |
| `printix_jobs_stuck(minutes)` | Jobs older than N minutes still in queue. |
| `printix_print_trends(group_by, days)` | Print-volume time series (day/week/month). |
| `printix_cost_by_department(department_field, days, cost_per_mono, cost_per_color)` | Cost aggregation by user-attribute field. |
| `printix_compare_periods(days_a, days_b, offset_b)` | Compares two equal-length windows (e.g. "last 30 days vs. the 30 before"). |

---

## Access & Governance

*(new in v6.7.107)*

| Tool | Description |
| --- | --- |
| `printix_list_admins()` | All SYSTEM_MANAGER / SITE_MANAGER / KIOSK_MANAGER accounts. |
| `printix_permission_matrix()` | User × Group matrix (groups drive printer access in Printix). |
| `printix_inactive_users(days)` | Users with no activity in last N days (cleanup candidates). |
| `printix_sso_status(email)` | Entra/Azure SSO link status for a user. |

---

## Agent Workflow Helpers

*(new in v6.7.107)*

| Tool | Description |
| --- | --- |
| `printix_explain_error(code_or_message)` | Plain-language explanation + fix suggestions for known error codes. |
| `printix_suggest_next_action(context)` | Heuristic advisor: "given this situation, try these tools". |
| `printix_send_to_user(user_email, file_url \| file_content_b64, filename, target_printer, copies)` | High-level composite: resolve printer → submit → upload → complete → change owner. |
| `printix_onboard_user(email, display_name, role, pin, password, groups)` | Full onboarding workflow in one call. |
| `printix_offboard_user(email, force)` | Leaver flow: delete cards, clear local mappings, delete account. |

---

## Quality of Life

*(new in v6.7.107)*

| Tool | Description |
| --- | --- |
| `printix_whoami()` | Current tenant context + configured OAuth scopes. Debug helper. |
| `printix_quick_print(recipient_email, file_url, filename)` | One-shot send-to-user using first available printer. |
| `printix_resolve_printer(name_or_location)` | Fuzzy-match a printer by name/location/model → `printer_id:queue_id`. |
| `printix_natural_query(question)` | Maps a natural-language question to the right reporting tools. |

---

## Conventions

- **Return format:** every tool returns a JSON-encoded string ( `json.dumps` ).
- **Error shape:** Printix API errors come back as `{"error": true, "status_code": N, "message": "...", "error_id": "..."}`. Other errors as `{"error": "message"}`.
- **Tenant isolation:** each request resolves its tenant via Bearer token. Tools operate only on that tenant's data.
- **Pagination:** most listing tools accept `page` (0-based) and `page_size` / `size` (default 50, max typically 200).
- **Date format:** `YYYY-MM-DD` for all date arguments.
- **Cost parameters:** `cost_per_mono`, `cost_per_color`, `cost_per_sheet` are per page/sheet in the tenant's currency; defaults are rough EU averages.

## Integration Patterns

### "What cards does Marcus have?"

```
printix_find_user("marcus")
  → user_id
printix_get_user_card_context(user_id)
  → cards + enriched local data
```

### "Who printed the most last month?"

```
printix_top_users(days=30, limit=10, metric="pages")
```

### "User X can't print — help me debug"

```
printix_diagnose_user("x@company.com")
  → findings list with severities + suggestions
```

### "Onboard a new employee"

```
printix_onboard_user(email=..., display_name=..., groups="grp1,grp2")
printix_generate_id_code(user_id)    # optional
# card registration happens in iOS/Web client
```

### "How did print volume change vs last month?"

```
printix_compare_periods(days_a=30, days_b=30)
```

---

## AI Workflow Tools (v6.8.x — Phase 1)

High-level composition tools that combine multiple low-level steps into one AI-friendly call. Auto-PDL conversion (PDF → PCL XL via Ghostscript) is built in for every print path; default `pdl="auto"` (= PCL XL color), override with `pdl="passthrough"`, `"PCLXL"`, `"PCL5"`, `"POSTSCRIPT"`.

### Native File Ingest

- **`printix_print_self(file_b64, filename, ..., pdl="auto", color=True)`** Print a file to the calling MCP user's *own* secure-print queue. Self-user is resolved via `current_tenant.email`. Returns `{ok, job_id, owner_email, size_input, size_after_conversion, pdl, ...}`. → Killer use case: AI generates a PDF inline and queues it on the caller's printer.
- **`printix_send_to_capture(profile, file_b64, filename, metadata_json="{}")`** Push a file directly into a capture workflow (Paperless-ngx, etc.) — same code path as a webhook but without the printer/Azure-Blob detour. Calls `plugin.ingest_bytes()` directly. Async tool.
- **`printix_describe_capture_profile(profile)`** Returns the plugin schema (config fields + types + accepted metadata fields). Use *before* `send_to_capture` so the AI can build the right `metadata_json`.

## Multi-Recipient Print

- `printix_get_group_members(group_id_or_name)` Members of a Printix group (UUID or name; case-insensitive). Falls back to `_links.users` HAL link when present.
- `printix_get_user_groups(user_email_or_id)` Reverse lookup: which groups is user X in?
- `printix_resolve_recipients(recipients_csv)` Diagnostic tool. Resolves a mixed list — emails, `group:Marketing`, `entra:<oid>`, `upn:...` — to a flat printix-user list. Returns `{resolved, not_found, ambiguous}` *without* printing.
- `printix_print_to_recipients(recipients_csv, file_b64, filename, ..., fail_on_unresolved=True, pdl="auto", color=True)` One PDF, multiple recipients (one job per recipient). Conversion runs *once* before the loop.

## Onboarding & Time-Bombs

- `printix_welcome_user(user_email, template="default", auto_print_to_self=True, timebombs="card_enrol_7d,first_print_reminder_3d")` Onboarding companion: generates a personalized welcome PDF, optionally queues it on the new user, and arms time-bombs (deferred reminders with condition checks).
- `printix_list_timebombs(user_email="", status="pending")` List active/historical time-bombs.
- `printix_defuse_timebomb(bomb_id, reason="manual")` Manually defuse a time-bomb. Tenant-scoped.
- `printix_sync_entra_group_to_printix(entra_group_oid, printix_group_id="", sync_mode="report_only")` Microsoft Graph ( `Group.Read.All` ) → diff vs Printix group. Default read-only.

## Bonus

- `printix_card_enrol_assist(user_email, card_uid_raw, profile_id="")` AI onboarding: UID → profile transform → register in one call.
- `printix_describe_user_print_pattern(user_email, days=30)` Top printers, color quote, average page count.
- `printix_session_print(user_email, file_b64, filename, expires_in_hours=24)` Print job + auto-expire time-bomb.
- `printix_quota_guard(user_email="", window_minutes=5, max_jobs=10)` Pre-flight burst check (verdict `allow` / `throttle` / `block` ).
- `printix_print_history_natural(user_email="", when="today", limit=50)` Print history with natural-language windows: `today` , `yesterday` , `this_week` , `last_month` , `Q1` – `Q4` , `7d` , `30d` , ...

## Time-Bomb Engine — DB & Scheduler

A new table `user_timebombs` is created idempotently on first call. Columns: `id, tenant_id, user_id, user_email, bomb_type, trigger_at, action_json, status, created_at, resolved_at, last_message`.

An APScheduler job `timebomb_tick` runs hourly ( `minute=7` ) and:

1. Loads pending bombs with `trigger_at <= now`.
2. Re-checks the original condition (e.g. "*user has no card enrolled*").
3. If condition still holds → executes the action (queues a reminder PDF, writes audit log, etc.). Marks `fired`.
4. If condition no longer holds (user did the thing meanwhile) → marks `defused`. Auto-cleanup, no false reminders.

## Server-Side PDL Conversion

`src/print_conversion.py` ships with:

- `detect_pdl(file_bytes)` — magic-byte detection (PDF / PostScript / PCL5 / PCL XL / TEXT)
- `pdf_to_pclxl(pdf_bytes, color=True)` via Ghostscript `pxlcolor` / `pxlmono`
- `pdf_to_pcl5(pdf_bytes)` via `cdjcolor` / `ljet4`
- `pdf_to_postscript(pdf_bytes)` via `ps2write`
- `text_to_postscript(text)` for plaintext input
- `prepare_for_print(file_bytes, target="PCLXL", color=True)` — main entry, auto-routes based on detected source PDL

Ghostscript is installed in the container (Dockerfile +25 MB). Errors are wrapped in `ConversionError` and surfaced cleanly to the AI tool — no silent submits with broken bytes.

---

*Generated for Printix MCP Server v6.8.10. **127 tools** in total — 16 new in v6.8.x (see AI Workflow Tools section). For API reference of the underlying Printix Cloud API see printix.net/developer.*